МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · MISC

CLASSIC

Восьмеричное кодирование сигнатурами носителей в цепочке полиглотов с многоканальной сборкой ключа

MISC

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

MISC
КАТЕГОРИЯ

16
СЛАЙДОВ В РАЗБОРЕ

01

ПОСТАНОВКА ЗАДАЧИ

Выдан единственный файл `classic` без расширения. Наименование задания указывает на классическую вложенную структуру типа «архив в архиве», предполагающую последовательное извлечение до достижения внутреннего слоя.

Первичная идентификация даёт противоречивый результат:

```
file classic
# classic: BMP image data
```

Утилита `file` определяет файл как растровое изображение, тогда как наименование указывает на архивную структуру. Проверка на архив:

```
unzip -l classic
# warning: ... extra bytes at beginning or within zipfile
# ... 1 file
```

Файл одновременно является корректным изображением и корректным архивом. Данная структура представляет собой полиглот: в начале расположен носитель с соответствующей сигнатурой, в конце — архив, поскольку формат ZIP считывается от завершающей записи каталога. Каждый слой содержит носитель некоторого формата с присоединённым архивом, содержащим следующий слой.

CLASSIC
MISC

01

ПОСТАНОВКА ЗАДАЧИ

ПРОДОЛЖЕНИЕ

```
$ python -c "d=open('classic','rb').read(); i=d.find(b'PK\x03\x04')"

magic носителя      : 42 4d 1e 00
размер файла        : 156826
zip начинается с    : offset 10 -> перед ним 10 байт носителя

file classic        : PC bitmap, Windows 3.x format
unzip -l classic    : warning: extra bytes at beginning, 1 file
```

Рис. 1. Структура полиглота: носитель с сигнатурой в начале файла и архив в его завершающей части.

CLASSIC
MISC

02

РАЗВОРАЧИВАНИЕ ВЛОЖЕННОЙ СТРУКТУРЫ

CLASSIC
MISC

Извлечение первого слоя обнаруживает носитель иного формата, также содержащий присоединённый архив.

```
mkdir x && cd x && unzip ../classic
file _
# _: PC bitmap, Windows 3.x format
```

Ввиду большого числа слоёв ручное извлечение нецелесообразно. Существенным обстоятельством является смещение внутренних оффсетов архива вследствие присоединения носителя перед ним; библиотека `zipfile` корректирует это смещение автоматически.

```
import io, zipfile

def unwrap_once(data: bytes) -> bytes:
    z = zipfile.ZipFile(io.BytesIO(data))   # prepend-оффсеты правятся автоматически
    return z.read(z.namelist()[0])          # на каждом слое ровно одна запись

data = open("classic", "rb").read()
depth = 0
while True:
    try:
        data = unwrap_once(data)
        depth += 1
    except zipfile.BadZipFile:
        break
```

02

РАЗВОРАЧИВАНИЕ ВЛОЖЕННОЙ СТРУКТУРЫ

ПРОДОЛЖЕНИЕ

CLASSIC
MISC

Процедура завершается на глубине 552, причём внутренний слой содержит не искомое значение, а текстовую строку `not classic, try more`.

```
... всего слоёв: 552
терминатор       -> dump\terminator.bin  (первые 22 байта = текст)
```

Рис. 2. Процесс автоматического разворачивания вложенных слоёв.

```
not classic, try more

00000016  ee 06 a3 59 ca 5e 2a 21 df 03 dc 24 bf 0c a4 3b  ...Y.^*!...$...;
00000026  8c 9b c6 1c 90 20 5a e8 12 e8 5c 7b 8f 73 5a 7a  ..... Z...\{.sZz
00000036  29 44 0f fe 8e f9 d7 ca c5 2c 9f ee 68 10 9c b5  )D.......,..h...
00000046  2c a8 91 16 47 22 38 b7 85 23 bb 09 42 e8 ff     ,...G"8..#..B..

за строкой идёт бинарный блок: 63 байт
```

Рис. 3. Достижение внутреннего слоя, содержащего строку-указание вместо ответа.

03

ПЕРЕИНТЕРПРЕТАЦИЯ СТРУКТУРЫ

Внутренний слой содержит указание на то, что классический подход, ориентированный на извлечение единственного вложенного элемента, неверен. Проверка носителей на наличие скрытых данных методами анализа печатных строк, младших битов и присоединённых после завершения данных результата не даёт: носители по отдельности не содержат посторонней информации.

Следовательно, значимой является не содержимое отдельного слоя, а структура последовательности слоёв в целом: формат носителя на каждом уровне и порядок их следования.

CLASSIC
MISC

04

ВОСЬМЕРИЧНОЕ КОДИРОВАНИЕ ФОРМАТАМИ

CLASSIC
MISC

Перечень форматов носителей по глубине содержит ровно восемь различных типов:

```
jpg  png  gif  bmp  pdf  wav  mp3  ogg
```

Число различных форматов в сочетании с указанием на неклассический характер задачи определяет систему счисления: каждый слой кодирует одну восьмеричную цифру, задаваемую форматом носителя. Цифра определяется по сигнатуре в начале файла, а не по маркеру архива, присутствующему одинаково на всех слоях.

```
SIG = {
    b"\xFF\xD8\xFF":       0,
    b"\x89PNG\r\n\x1a\n":  1,
    b"GIF89a":             2,
    b"BM":                 3,
    b"%PDF-":              4,
    b"RIFF":               5,
    b"ID3":                6,
    b"OggS":               7,
}
def digit(head: bytes):
    for sig, d in sorted(SIG.items(), key=lambda kv: -len(kv[0])):
        if head.startswith(sig):
            return d
    return None
```

04

ВОСЬМЕРИЧНОЕ КОДИРОВАНИЕ ФОРМАТАМИ

ПРОДОЛЖЕНИЕ

CLASSIC
MISC

Проход по структуре с извлечением цифры на каждом слое даёт последовательность из 552 восьмеричных цифр; внутренний текстовый слой служит терминатором.

depth	digit	format
0	3	bmp
1	3	bmp
2	4	pdf
3	6	mp3
4	2	gif
5	4	pdf
6	4	pdf
7	0	jpg
8	2	gif
9	0	jpg
10	6	mp3
11	4	pdf
12	6	mp3
13	1	png
14	0	jpg
15	1	png
16	2	gif
17	4	pdf
18	6	mp3
19	5	wav
20	1	png
21	5	wav
22	1	png
23	1	png
24	2	gif
25	0	jpg
26	6	mp3
27	2	gif
28	0	jpg
29	1	png
30	4	pdf
31	1	png
32	1	png
33	0	jpg
34	0	jpg
35	5	wav
36	0	jpg
37	1	png
38	1	png
39	7	ogg
40	2	gif

Рис. 4. Извлечение последовательности восьмеричных цифр из форматов носителей.

05

ПРЕОБРАЗОВАНИЕ ПОСЛЕДОВАТЕЛЬНОСТИ В ТЕКСТ

Последовательность из 552 восьмеричных цифр соответствует 1656 битам, то есть 207 байтам. Целочисленность результата подтверждает гипотезу об осмысленном содержании.

```
bits = "".join(f"{d:03b}" for d in digits)
text = bytes(int(bits[i:i+8], 2) for i in range(0, len(bits), 8))
```

Результат содержит осмысленное начало и заполняющий остаток:

```
ne CLASSIC a POLYGLOT. sloy 410=JPG chitaetsya na tempe iz zvuka;
sloy 470=MP3 BPM=seed ID3=Ka; klyuch SHA256(mp3,jpg); sunduk v terminatore. #<набивка>
```

Размещение инструкции внутри канала, образованного форматами, исключает её непосредственное извлечение поиском по файлу: прочтение возможно лишь после полного прохода канала. Заполняющая часть после разделителя служит выравниванием до требуемой длины.

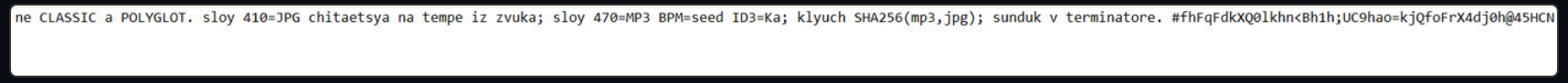


Рис. 5. Извлечённая инструкция, определяющая схему сборки ключа.

CLASSIC
MISC

06

ИНТЕРПРЕТАЦИЯ ИНСТРУКЦИИ

Инструкция называет два слоя по их номерам и описывает схему сборки. Слой 410 формата JPEG считывается с использованием темпа, извлекаемого из звука. Слой 470 формата MP3 содержит темп в качестве инициализирующего значения и половину ключа в теге ID3. Ключ вычисляется как SHA-256 от конкатенации данных из обоих слоёв. Контейнер размещён в терминаторе, ранее принятом за тупиковый элемент.

Проход по структуре модифицируется с сохранением слоёв для доступа по номеру; вводится функция отделения носителя от присоединённого архива.

```
def collect(path):
    data, layers, depth = open(path, "rb").read(), {}, 0
    while digit(data[:8]) is not None:
        layers[depth] = data
        data = unwrap_once(data)
        depth += 1
    return layers, data              # data на выходе это терминатор

def carrier(layer: bytes) -> bytes:  # носитель без хвостового zip
    return layer[:layer.find(b"PK\x03\x04")]
```

CLASSIC
MISC

07

ИЗВЛЕЧЕНИЕ ДАННЫХ ИЗ АУДИОНОСИТЕЛЯ

CLASSIC
MISC

Носитель слоя 470 представляет собой короткий метрономический сигнал. Половина ключа извлекается из тега:

```
from mutagen.id3 import ID3
open("s470.mp3", "wb").write(carrier(layers[470]))
Ka = bytes.fromhex(ID3("s470.mp3").getall("TXXX")[0].text[0])
```

Инструкция подчёркивает, что темп извлекается из звукового сигнала, а не из метаданных. Применение стандартного детектора темпа даёт дробное и нестабильное значение вследствие встроенного байесовского приора, смещающего оценку к типичным значениям. Для использования в качестве инициализирующего значения требуется целое и воспроизводимое число, вследствие чего темп измеряется непосредственно по геометрии импульсов: определяются моменты их начала, вычисляется медиана межимпульсных интервалов и выполняется перевод в удары в минуту.

```
import numpy as np, librosa
def measure_tempo(path, sr=22050):
    y, _ = librosa.load(path, sr=sr)
    onsets = librosa.onset.onset_detect(y=y, sr=sr, units="time", backtrack=False)
    return int(round(60.0 / np.median(np.diff(onsets))))
P = measure_tempo("s470.mp3")
```

07

ИЗВЛЕЧЕНИЕ ДАННЫХ ИЗ АУДИОНОСИТЕЛЯ

ПРОДОЛЖЕНИЕ

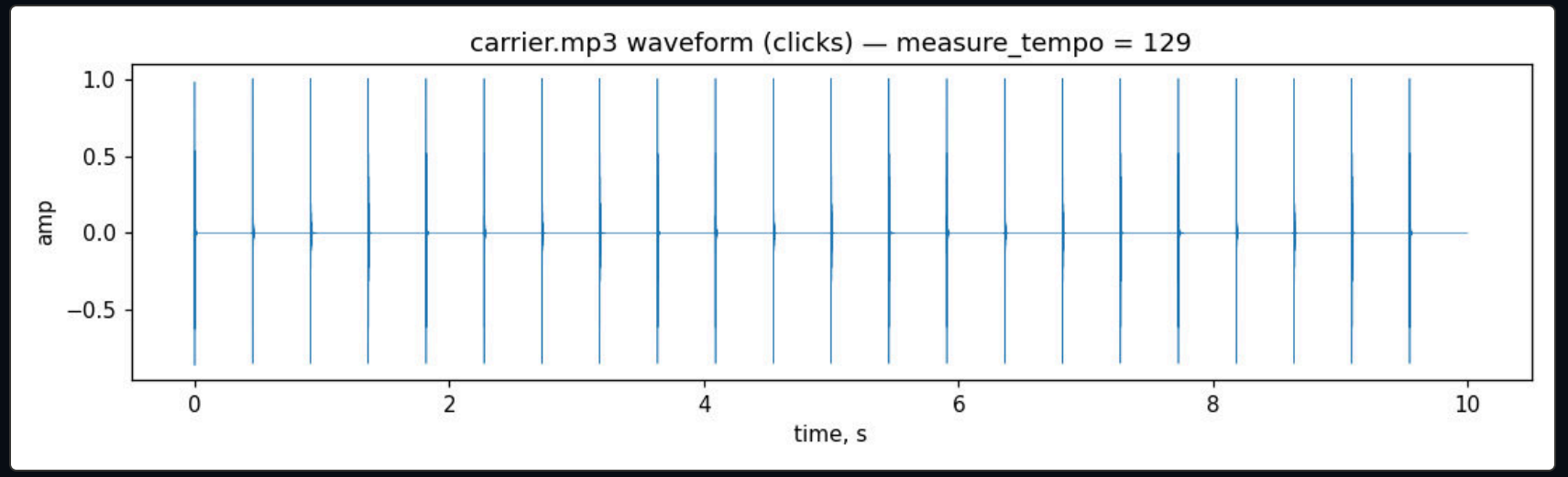


Рис. 6. Метрономический характер аудионосителя, обеспечивающий устойчивое измерение темпа.

```
P (measure_tempo)    = 129
Ka (ID3 TXXX)        -> dump\Ka.hex   (f7ab99704c47a7b054ff8a7e168b00e7)
```

Рис. 7. Извлечение половины ключа из тега и измеренного темпа.

CLASSIC
MISC

08

ИЗВЛЕЧЕНИЕ ДАННЫХ ИЗ ИЗОБРАЖЕНИЯ

CLASSIC
MISC

Измеренный темп служит инициализирующим значением для стеганографического извлечения из изображения слоя 410.

Существенным обстоятельством является размещение данных в квантованных коэффициентах дискретного косинусного преобразования, а не в отсчётах изображения: декодирование в пиксели с последующим перекодированием изменяет коэффициенты и разрушает вложение. Обработка ведётся на уровне коэффициентов. Данные размещены в младшем бите одного среднечастотного коэффициента, а порядок обхода блоков задаётся генератором псевдослучайных чисел с инициализирующим значением, равным темпу.

```
import jpeglib
def dct_extract(path, P, nbytes):
    Y = jpeglib.read_dct(path).Y
    bh, bw = Y.shape[0], Y.shape[1]
    nb = nbytes * 8
    order = np.random.RandomState(P).permutation(bh * bw)[:nb]
    bits = "".join(str(int(Y[by, bx, 2, 3]) & 1)
                   for by, bx in (divmod(o, bw) for o in order))
    return bytes(int(bits[k:k+8], 2) for k in range(0, nb, 8))

open("s410.jpg", "wb").write(carrier(layers[410]))
Kb = dct_extract("s410.jpg", P, 16)
```

09

СБОРКА КЛЮЧА И ИЗВЛЕЧЕНИЕ СОДЕРЖИМОГО

CLASSIC
MISC

Обе половины ключа определены. Ключ вычисляется как SHA-256 от их конкатенации; контейнер размещён в терминаторе, за видимой текстовой строкой которого следует бинарный блок.

```
import hashlib
from Crypto.Cipher import AES

key = hashlib.sha256(Ka + Kb).digest()
C = terminator.split(b"not classic, try more\n", 1)[1]
nonce, tag, ct = C[:12], C[12:28], C[28:]
flag = AES.new(key, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)
```

Успешная проверка имитовставки в режиме GCM подтверждает корректность собранного ключа.

```
Kb (DCT, seed=P)    -> dump\Kb.hex   (1cfbcadc28d5f3c6266ec18481876a16)
key=SHA256(Ka||Kb)  -> dump\key.hex
C  (шифроблок)      -> dump\C_ciphertext.bin
ФЛАГ                -> dump\FLAG.txt

>>> flag{3sc4p3_th3_s4ndb0x_2_inn0v4t3} <<<

00000000  ee 06 a3 59 ca 5e 2a 21 df 03 dc 24 bf 0c a4 3b  ...Y.^*!...$...;
00000010  8c 9b c6 1c 90 20 5a e8 12 e8 5c 7b 8f 73 5a 7a  ..... Z...\{.sZz
00000020  29 44 0f fe 8e f9 d7 ca c5 2c 9f ee 68 10 9c b5  )D.......,..h...
```

Рис. 8. Расшифрование контейнера, размещённого в элементе, изначально принятом за тупиковый.

10

ЗАКЛЮЧЕНИЕ

Решение опирается на четыре обстоятельства. Первое состоит в распознавании структуры как цепочки полиглотов, а не простой вложенности архивов. Второе заключается в переинтерпретации, при которой значимой оказывается последовательность форматов, а не содержимое отдельных слоёв, причём число различных форматов задаёт систему счисления. Третье состоит в извлечении инструкции через восьмеричный канал, что исключает её обнаружение прямым поиском. Четвёртое определяется многоканальной сборкой ключа с измерением темпа по геометрии сигнала вместо метаданных и извлечением из коэффициентов преобразования вместо отсчётов изображения.

Итоговый результат: `flag{3sc4p3_th3_s4ndb0x_2_inn0v4t3}`

CLASSIC
MISC

ФЛАГ

```
flag{3sc4p3_th3_s4ndb0x_2_inn0v4t3}
```

Задание «Classic» решено